Aran Learn 9: Transformers and Zero-Shot NER

For: Bharadhwaj and Arun (roadmap items 7 and 8). Inbathamizhan and Aasif read sections 1, 7 and 8 so they can explain the design. **Time:** about 8 hours with practice. **Before this:** Learn 5 in full, especially section 3 (BIO tags, CRF) and section 3.4 (transformers, the next step).

Aran's name finder is a CRF (Learn 5). It is fast and small, and it reaches F1 0.85 on English news, but end to end it removes only 74% of names from log lines, and it can only find the entity types it was trained on. Transformer models read whole sentences at once and can be asked for new entity types by name. This doc explains how they work from the ground up, how to use them for NER in English, Hindi and Tamil, and how to keep them fast enough for a bank's log volume.

Outputs in sections 2 to 6 and 8 were produced by running the code shown. Section 5 downloads a model from Hugging Face, which our build machine could not reach, so its output is described rather than pasted; run it yourself and write your own numbers into your notes (practice P3).

1. Why move past the CRF

Problem with the CRF today	How a transformer helps
It looks at a small window of hand-made features (word shape, the word before and after)	Every word can look at every other word in the line
A name it never saw in training, in a phrasing it never saw, is often missed	Pre-training on billions of words means it has seen most names and phrasings already
New entity types (nominee, employer, branch) need new labelled data	Zero-shot models take the type's name as input and find it with no new training
Separate models per language	One multilingual model covers English, Hindi, Tamil and mixes of them

The cost is speed and size: a CRF tags thousands of lines a second on one CPU core, a transformer tens to hundreds. Section 7 is about paying that cost only where it is needed.

2. Turning words into numbers: embeddings

A model can only do arithmetic, so every word (or piece of a word) becomes a list of numbers called an **embedding** or **vector**. Training arranges these vectors so that words used in similar ways get similar vectors.

How similar two vectors are is measured by **cosine similarity**: 1 means pointing the same way, 0 means unrelated.

```
import numpy as np
words = ["Anna", "Nagar", "branch"]
# toy 4-number embeddings (made up for teaching; real ones have 768 numbers)
X = np.array([[1.0, 0.2, 0.0, 0.5],
              [0.9, 0.1, 0.1, 0.6],
              [0.0, 1.0, 0.8, 0.0]])
```

```
def cos(a, b): return a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(cos(X[0], X[1]), cos(X[0], X[2]))
```

```
cos(Anna, Nagar) = 0.98  cos(Anna, branch) = 0.14
```

In this toy, "Anna" and "Nagar" point almost the same way because they often appear together in place names; "branch" is a different kind of word.

3. Splitting words: subword tokenisation

A model cannot have a vector for every possible word. Names especially are endless: Priya, Priyanka, Priyadharshini. So a **tokeniser** splits text into **subwords** from a fixed vocabulary of, typically, 30,000 to 250,000 pieces. Common words stay whole; rare words are built from pieces.

The usual method is **BPE** (byte-pair encoding): start from single characters, then repeatedly merge the most frequent neighbouring pair into a new piece, until the vocabulary is full. You can train a tiny one in seconds with the `tokenizers` library (`pip install tokenizers`):

```
from tokenizers import Tokenizer, models, trainers, pre_tokenizers
corpus = [
    "customer name Priya Raman city Madurai",
    "customer name Ramesh Kumar city Chennai",
    "applicant Suresh Raman address Anna Nagar Chennai",
    "aapka otp hai kisi ko na batayen",
    "ग्राहक का नाम प्रिया रमन है",
    "ग्राहक का पता अन्ना नगर चेन्नई है",
    "வாடிக்கையாளர் பெயர் பிரியா ராமன்",
    "வாடிக்கையாளர் முகவரி அண்ணா நகர் சென்னை",
] * 20
tok = Tokenizer(models.BPE(unk_token="[UNK]"))
tok.pre_tokenizer = pre_tokenizers.Whitespace()
tok.train_from_iterator(corpus, trainers.BpeTrainer(vocab_size=200, special_tokens=["
[UNK]"]))
for s in ["customer Priyanka Ramanathan", "ग्राहक प्रियंका", "வாடிக்கையாளர் பிரியங்கா"]:
    e = tok.encode(s)
    print(s, "->", e.tokens, "word_ids", e.word_ids)
```

```
customer Priyanka Ramanathan -> ['customer', 'Priy', 'an', 'ka', 'Ra', 'm', 'a', 'na', 't',
'h', 'an'] word_ids [0, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2]
ग्राहक प्रियंका -> ['ग्राहक', 'प्र', 'ि', 'य', '[UNK]', 'का'] word_ids [0, 1, 1, 1, 1, 1]
வாடிக்கையாளர் பிரியங்கா -> ['வாடிக்கையாளர்', 'பி', 'ரி', 'ய', '[UNK]', 'ं', 'க', 'ா'] word_ids
[0, 1, 1, 1, 1, 1, 1, 1]
```

What this shows:

- "customer" and "ग्राहक" (customer) were common in the corpus, so they stay whole.
- "Priyanka" was never seen, but it is built from known pieces, so a model can still guess it is a name.
- The Hindi anusvara (ं) and the Tamil letter ங never appeared in our tiny corpus, so they became `[UNK]` (unknown), and the information is lost. Real multilingual tokenisers avoid this by falling back to raw bytes, and by being trained on a lot of Indian-language text. **This is why the choice of model matters for Indian text**: a tokeniser trained mostly on English

chops Tamil into many tiny pieces, which makes the model slower and worse. MuRIL and IndicBERT were trained with Indian languages in mind.
- `word_ids` says which original word each piece came from. Section 4.2 needs it.

4. Attention: how a transformer reads a line

4.1 The idea

In "Anna Nagar branch", the word "Anna" could be a person's name or part of a place. A transformer decides by letting each word **attend** to the others: it computes how relevant every other word is, then mixes their vectors in, weighted by that relevance. After this step, the vector for "Anna" carries information from "Nagar", and the model can tell it is a place.

The simplest version, using the toy vectors from section 2:

```
scores = X @ X.T / np.sqrt(X.shape[1])                                  # how well each pair
matches
w = np.exp(scores) / np.exp(scores).sum(axis=1, keepdims=True)       # softmax: each row sums
to 1
new_nagar = w[1] @ X                                                    # Nagar's new vector
```

```
attention weights (row = word looking, column = word looked at):
  Anna    [0.39 0.38 0.23]
  Nagar   [0.39 0.38 0.23]
  branch  [0.25 0.24 0.51]
new vector for Nagar = [0.73 0.35 0.22 0.42]
```

"Nagar" takes 39% of its new vector from "Anna" and only 23% from "branch". "branch" mostly keeps its own meaning (51%).

A real transformer does three more things: it first multiplies each vector by three learned matrices to get a **query** (what am I looking for), a **key** (what do I offer) and a **value** (what do I pass on), and uses query-times-key for the scores; it runs many of these **attention heads** side by side, each learning a different kind of relation; and it stacks 12 or more such **layers**. The core step is still the one above.

4.2 Pre-training and fine-tuning

- **Pre-training.** The model reads billions of sentences with some words hidden and learns to guess them. This teaches it language in general. Google, AI4Bharat and others have done this already and published the results for free.
- **Fine-tuning.** We add a small output layer that predicts a BIO tag for each piece (Learn 5, section 3.1) and train the whole model briefly on our labelled NER data.

One detail trips everyone up: labels are per word, but the model sees pieces. The usual fix is to give the label to the first piece of each word and tell training to ignore the rest, using the special value `-100`:

```
word_labels = ["O", "B-PER", "I-PER"]          # customer Priyanka Ramanathan
word_ids = [0, 1, 1, 1, 2, 2, 2, 2, 2, 2]     # from the tokenizer
labels, prev = [], None
for wid in word_ids:
    labels.append(word_labels[wid] if wid != prev else -100)   # -100 = ignore in the loss
```

```
    prev = wid
print(labels)
```

```
['O', 'B-PER', -100, -100, 'I-PER', -100, -100, -100, -100, -100, -100]
```

At prediction time, read the tag of each word's first piece and ignore the others.

4.3 Models worth trying for Aran

Model (Hugging Face name)	What it is	Use in Aran
`google/muril-base-cased`	BERT pre-trained on 17 Indian languages, including transliterated text	Fine-tune for Hinglish and Tanglish logs
`ai4bharat/IndicNER`	Already fine-tuned for person, location and organisation in 11 Indian languages	Use directly for Hindi and Tamil names, no training needed
`xlm-roberta-base`	100 languages	General baseline to compare against
`knowledgator/gliner-pii-base-v1.0` and other GLiNER-PII models	Zero-shot, trained for personal data types	Section 5

5. Zero-shot NER with GLiNER

5.1 The idea

A normal NER model has a fixed list of tags baked into its output layer. **GLiNER** (Zaratiana et al., NAACL 2024) works differently: you pass the entity types you want as plain words, such as "person", "nominee name" or "home address". The model turns each type name into a vector, turns every possible span of the text into a vector, and scores how well each span matches each type. Any span scoring above a threshold is returned.

So adding a new type is a one-word change, not a new dataset. Several GLiNER models have been trained specifically on personal data (PII), which suits Aran.

5.2 Code

On your own laptop or Google Colab (`pip install gliner`):

```
from gliner import GLiNER
model = GLiNER.from_pretrained("knowledgator/gliner-pii-base-v1.0")
text = "applicant Suresh Gill requested address change to 12 Anna Nagar, Chennai; nominee
Lakshmi Gill"
labels = ["person", "address", "nominee name"]
for e in model.predict_entities(text, labels, threshold=0.5):
    print(e["label"], "|", e["text"], "|", round(e["score"], 2), e["start"], e["end"])
```

Each result gives the type, the text, a score between 0 and 1, and the character positions, which is exactly what Aran's detector needs: the positions tell the firewall what to replace, and the score becomes one more piece of evidence for the calibrated detector (Learn 4 and Learn 5 section 5). Note that GLiNER's score is not a calibrated probability; it must go through the same calibration step as every other model before its numbers mean anything.

What to look for when you run it. Whether "Lakshmi Gill" comes out as "nominee name", "person", or both; whether the address span includes "12" and "Chennai"; and how the result changes at thresholds 0.3, 0.5 and 0.7. Then try the same sentence in Hindi and Tamil. Write all of it down (practice P3); these are the first numbers for roadmap item 7.

5.3 Limits

- Type names matter. "person" and "customer name" may give different results; test a few wordings on validation data and keep the best.
- It is slower than a fixed-tag model, because it scores many spans.
- It has not seen Indian bank log formats. Measure it on Aran's held-out templates and the 16,000 real Loghub lines before trusting it, exactly as E5 and E9 did for the CRF.

6. Making it fast enough

A bank's logs run to millions of lines an hour. Aran's firewall does 299 lines per second today in pure Python, and NER is already the slowest stage. Three tools keep a transformer from making that worse.

6.1 The cascade: only run NER where names can be

Most log lines are machine text: timestamps, paths, key=value pairs, stack traces. Names live in free-text lines such as agent notes and complaint text. A cheap gate decides which lines go to the expensive model:

```
import unicodedata

def is_word(tok: str) -> bool:
    """True if every character is a letter or a vowel sign (category L* or M*), any
script."""
    tok = tok.strip(".,:;!?()\"'")
    return len(tok) >= 2 and all(unicodedata.category(c)[0] in "LM" for c in tok)

def is_free_text(line: str) -> bool:
    toks = line.split()
    words = [t for t in toks if is_word(t)]
    return len(words) >= 4 and len(words) / len(toks) >= 0.6
```

```
False | 081109 203615 148 INFO dfs.DataNode$PacketResponder: PacketR
False | 2026-10-14 10:02:11 txn=TXN88231 amt=500.00 status=OK
True  | applicant Suresh Gill requested address change to 12 Anna Na
True  | agent note: customer Priya Raman called about card block
True  | ग्राहक प्रिया रमन ने कार्ड ब्लॉक करने को कहा
True  | வாடிக்கையாளர் பிரியா ராமன் கார்டை முடக்கச் சொன்னார்
False | GET /api/v1/health 200 3ms
```

Why `unicodedata`: Hindi and Tamil vowel signs (like ि and ாி) are "marks" (category M), not letters, so a plain `str.isalpha()` test calls "प्रिया" not a word and the gate would skip every Indian-language line. A bug like this silently turns off NER for exactly the text that needs it, so test the gate in every script you support.

The gate has a known gap: `name=Priya` in a key=value line is skipped. That is fine, because the key name is caught by the context rules (Learn 8, section 2). Measure the gate on real data: what

share of Loghub lines it sends to NER (the speed saving), and what share of lines with injected names it lets through (the recall cost).

6.2 Distillation: a small student copies a big teacher

Train a big, accurate **teacher** model on your labelled data, use it to label a large pile of unlabelled text, and train a small, fast **student** on those labels. The student learns from far more examples than you could ever label by hand. Here is the effect on a generic task with scikit-learn, where 2,000 labelled examples and 38,000 unlabelled ones are available:

```
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier

X, y = make_classification(n_samples=60000, n_features=20, n_informative=8, random_state=0)
Xl, yl = X[:2000], y[:2000]            # small labelled set
Xu = X[2000:40000]                      # big unlabelled set (labels thrown away)
Xt, yt = X[40000:], y[40000:]           # test set

teacher = RandomForestClassifier(n_estimators=300, random_state=0).fit(Xl, yl)
plain = DecisionTreeClassifier(max_depth=8, random_state=0).fit(Xl, yl)
student = DecisionTreeClassifier(max_depth=8, random_state=0).fit(Xu, teacher.predict(Xu))
```

Model	Test accuracy	Speed on our machine (rows per second)
Teacher, 300-tree forest	0.958	about 60,000
Small tree trained on the 2,000 labels only	0.913	about 11,400,000
Same small tree, distilled from the teacher	0.937	about 10,900,000

The distilled student closed more than half the gap to the teacher while staying about 180 times faster. Your speeds will differ by machine; the pattern will not. For Aran the teacher would be GLiNER or a fine-tuned MuRIL, and the student the existing CRF, retrained on teacher-labelled log lines.

6.3 Smaller and faster models

- **Batching:** send 32 or 64 lines to the model at once instead of one by one.
- **ONNX Runtime:** export the model to ONNX format and run it with `onnxruntime`, usually several times faster on CPU than plain PyTorch.
- **Quantisation:** store weights as 8-bit integers instead of 32-bit floats. The model shrinks to about a quarter of the size and runs faster, usually with a small accuracy loss. Always re-measure F1 afterwards.

7. Indian languages and Tamil

- **Scripts.** Tamil, Devanagari and Latin all appear in Indian bank text, and people mix them ("customer romba angry, card block pannunga").
- **Transliteration.** Tanglish and Hinglish write Tamil and Hindi in Latin letters, with no fixed spelling. MuRIL was trained partly on transliterated text for this reason.
- **Data.** Naamapadam (Mhaske et al., ACL 2023, AI4Bharat) is the largest public NER dataset for Indian languages: 11 languages including Tamil, with person, location and organisation

tags. It was built automatically by projecting English tags onto translations, so expect some noise in the labels.
- **Our own templates.** As with English, never train on real customer text. Write Tamil and Tanglish bank-log templates in `synth.py` with invented names from public name lists, and keep separate templates for training and testing (Learn 5, section 1.3).

8. Evaluating fairly

The same rules as Learn 5, with two additions:

1. **Same test set for every model.** The CRF, IndicNER, fine-tuned MuRIL and GLiNER are all scored on identical held-out templates and the same 16,000 real Loghub lines, with `seqeval` entity-level F1.
2. **Report speed next to F1.** A table with lines per second on one CPU core, with and without the gate. A model that gains 5 F1 points but makes Aran ten times slower may be the wrong choice, and judges will ask.
3. **Report false alarms on real logs.** The Loghub lines contain no Indian names, so every name a model finds there is a false alarm. A transformer that sees "names" in Java class names is worse than it looks on F1 alone.

9. Practice

Use Google Colab (free GPU) for P3 to P5.

- [ ] **P1.** Run the tokeniser in section 3 with `vocab_size` 200, 500 and 1,000, and count how many pieces "Priyanka Ramanathan" splits into each time. Explain the trend.
- [ ] **P2.** Load the real MuRIL and XLM-R tokenisers (`AutoTokenizer.from_pretrained`) and count pieces for 20 Tamil words. Which model splits Tamil less?
- [ ] **P3.** Run the GLiNER code in section 5.2 on 10 lines of your own in English, Hindi and Tamil, at three thresholds, and record every result in a table.
- [ ] **P4.** Run `ai4bharat/IndicNER` on the Tamil part of Naamapadam's test split and report entity-level F1 for PER.
- [ ] **P5.** Fine-tune MuRIL on the same training data the CRF uses and compare F1 and lines per second against the CRF on identical test data.
- [ ] **P6.** Run the gate in section 6.1 over all 16,000 Loghub lines used in E9. What share goes to NER? Then over the 1,110 injected lines: what share of lines with names got through?

10. Further reading

- Jay Alammar, "The Illustrated Transformer" (free blog post): the clearest picture-by-picture explanation of attention.
- Hugging Face course, chapters 1 to 3 and the "Token classification" section of chapter 7 (huggingface.co/learn): fine-tuning for NER step by step.
- Vaswani et al., "Attention Is All You Need" (NeurIPS 2017), the original transformer paper.
- Devlin et al., "BERT" (NAACL 2019).
- Khanuja et al., "MuRIL: Multilingual Representations for Indian Languages" (2021).
- Mhaske et al., "Naamapadam: A Large-Scale Named Entity Annotated Data for Indic Languages" (ACL 2023).
- Zaratiana et al., "GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer" (NAACL 2024).

- Hinton, Vinyals and Dean, "Distilling the Knowledge in a Neural Network" (2015).